

Université d'Ottawa
Faculté de génie

École de science informatique
et de génie électrique



University of Ottawa
Faculty of Engineering

School of Electrical Engineer
and Computer Science

Cours: CEG3585
Semestre: Été 2023

Professor: Miguel Garzón
Room: STE 5026B
Phone: (613)562-5800 x 2129
Email: mgarzon@uottawa.ca

Lab 3 (3%)

Programmation de Sockets et le protocole IPv4

Travailler en groupes de deux ou individuellement

Semaine du 5 et 12 Juin

Date de soumission : 19 Juin (avant minuit)

Objectifs:

- Reconnaître l'importance du protocole IP
- Se familiariser avec la programmation *Network Socket*, qui sera nécessaire dans les laboratoires suivants.

Au cours de ce laboratoire, les étudiants doivent créer deux programmes utilisant des sockets pour transmettre des paquets IP d'un client à un serveur.

Quoi soumettre:

- Soumettez votre code (deux classes Java) avant le 22 juin (minuit).
- Une prise d'écran montrant la console du client et du serveur lors de l'exécution du scénario ci-dessous.
- Un fichier README indiquant comment exécuter le code. Incluez vos noms et numéros d'étudiants.

Le Scénario:

Client's IP: 192.168.0.3 (dans cet exemple, il variera en fonction de l'adresse IP de votre client)

Server's IP: 192.168.0.1 dans cet exemple, il variera en fonction de l'adresse IP de votre serveur)

UTILISATEUR

L'utilisateur invoque le programme comme suit (si Python est utilisé):

```
python packet_sender.py -server 192.168.0.1 -payload "COLOMBIA 2 - MESSI 0"
```

PROGRAMME CLIENT (Encodeur - packet_sender.py)

Convertit le texte en HEX:

434f 4c4f 4d42 4941 2032 202d 204d 4553 5349 2030

Encapsule les données dans un datagramme IP (paquet) et initialise la somme modulaire (**checksum**) à 0000

4500 0028 1c46 4000 4006 **0000** COA8 0003 COA8 0001 **434f 4c4f 4d42 4941 2032 202d 204d 4553 5349 2030**

Calcule le **checksum**:

4500 0028 1c46 4000 4006 **9D35** COA8 0003 COA8 0001 **434f 4c4f 4d42 4941 2032 202d 204d 4553 5349 2030**

et l'envoi au client.

PROGRAMME SERVEUR (Décodeur - packet_receiver.py)

*Reçoit le flux de données et imprime à l'écran les données reçues avec le message suivant:
Les données reçues de 192.168.0.3 sont COLOMBIA 2 - MESSI 0
Les données ont 160 bits ou 20 octets. La longueur totale du paquet est de 40 octets. La
vérification de la somme de contrôle confirme que le paquet reçu est authentique.*

Si le paquet reçu est corrompu, le message suivant doit être imprimé:

La vérification de la somme de contrôle montre que le paquet reçu est corrompu. Paquet jeté!

Plus de détails

Considérez l'information utilisé dans notre scénario ci-dessus:

4500 0028 1c46 4000 4006 **9D35** COA8 0003 COA8 0001 **434f 4c4f 4d42 4941 2032 202d 204d 4553 5349 2030**

- **45** correspond aux deux premiers champs de l'en-tête. 4 signifie IPv4 et 5 correspond à la longueur de l'en-tête. La longueur de l'en-tête étant décrite en mots de 4 octets, sa longueur réelle est de $5 \times 4 = 20$ octets. [fixe]
- **00** correspond au TOS ou au type de service. Cette valeur de TOS indiquait un fonctionnement normal. [fixe]
- **0028** correspond au champ de longueur totale de l'en-tête IP. Donc, dans ce cas, la longueur totale du paquet IP est de 40 octets. L'en-tête est composé de 20 octets, le payload comprend 20 octets. [variable]
- **1c46** correspond au champ d'identification. [fixe]
- **4000** peut être divisé en deux octets. Ces deux octets correspondent aux indicateurs et au décalage de fragment des champs d'en-tête IP. [fixe]

- **4006** peut être divisé en «40» et «06». Le premier octet «40» correspond au champ TTL et l'octet «06» correspond au champ de protocole de l'en-tête IP. «06» indique que le protocole est TCP. [fixe]
- **9D35** correspond à la somme de contrôle définie à l'extrémité source (qui a envoyé le paquet) [variable]
- Les octets suivants **C0A8 0003** (192.168.0.3) et **C0A8 0001** (192.168.0.1) correspondent à l'adresse IP source et à l'adresse IP de destination dans l'en-tête IP. [variable et dépend des adresses IP du client et du serveur]
- 434f 4c4f 4d42 4941 2032 202d 204d 4553 5349 2030 correspond au payload (20 octets dans ce cas). [Variable]

Pour plus de simplicité: tous les champs marqués comme [fixe] devraient être **Hardcoded**. Aucune option n'est incluse dans le paquet. La longueur du payload + la longueur de l'en-tête doivent être divisibles par 8. Si ce n'est pas le cas, un remplissage (padding) doit être effectué (ajouter des zéros au payload).

Comment la somme de contrôle (16 bits) est calculée - ENCODAGE:

Si nous considérons que le champ de somme de contrôle de l'en-tête est 0 et calculons le total des 16 bits, ce sera:

$$4500 + 0028 + 1c46 + 4000 + 4006 + \textcolor{red}{0000} + C0A8 + 0003 + C0A8 + 0001 = 262C8$$

Avec l'ajout du report (carry): $2 + 62C8 = 62CA$

Son complément en HEX devient: 9D35 (FFFF-62CA)

Notez que la somme de contrôle est calculée en utilisant uniquement les champs d'en-tête. Autrement dit, les octets du payload ne sont pas inclus dans le calcul.

Comment la somme de contrôle (16 bits) est vérifiée - DÉCODAGE:

$$4500 + 0028 + 1c46 + 4000 + 4006 + 9D35 + C0A8 + 0003 + C0A8 + 0001 = 2FFFD$$

Avec l'ajout du report (carry): $2 + \text{FFFFD} = \text{FFFF}$

Son complément en HEX devient: 0000 (Aucun erreur)

Instructions additionnelles:

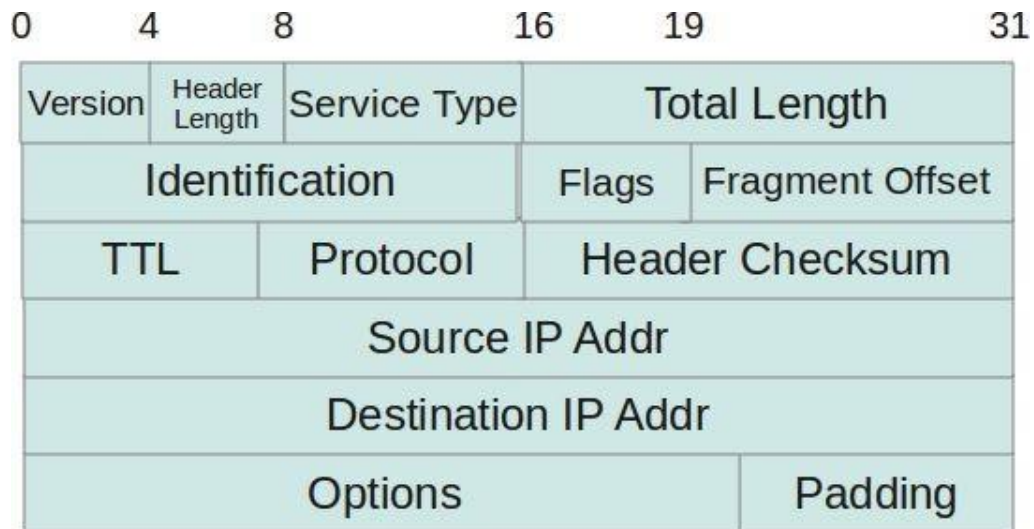
Vous devez implémenter deux programmes, un client et un serveur, et deux fonctions (méthodes), une pour encoder (côté client) et une pour décoder (côté serveur).

- Le client: PacketSender.java ou packet_sender.py
 - Envoie ce message encodé au serveur via socket
- Le serveur: PacketReceiver.java ou packet_receiver.py
 - Confirme au client que le message encodé a été reçu.
 - Décode le message et l'imprime à l'écran

Vous pouvez coder les deux programmes en utilisant Java 8+ ou Python 3.

Utilisez les fonctions intégrées nécessaires pour les conversions ASCII-> HEX.

Plus d'infos:



Python Sockets: <https://docs.python.org/2/howto/sockets.html>

Java Sockets: <https://docs.oracle.com/javase/tutorial/networking/sockets/clientServer.html>

Le barème d'évaluation utilisé par le TA sera le suivant:

Component	Points
1. Le client communique bien avec un serveur via un socket	20 points
2. L'encodage fonctionne correctement	10 points*
3. Le décodage fonctionne correctement	10 points*
4. Inclut un README qui décrit comment exécuter le code	5 points
5. Du côté du client, le message est encapsulé correctement.	5 points
Total	50 points

* Le TA exécutera cinq cas de test différents (avec des payloads différentes).
Assurez-vous que le scénario fourni fonctionne correctement.